KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade

Anand Nagar, Krishnankoil, Srivilliputtur (Via), Virudhunagar (Dt) - 626126, Tamil Nadu | info@kalasalingam.ac.in

SCHOOL OF COMPUTING

Department of Computer Science and Engineering

OPERATING SYSTEMS

Lab Record (212CSE3303)

Student Name : ……

Register Number : ……...

Section :……

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur (Via), Virudhunagar (Dt) - 626126, Tamil Nadu | info@kalasalingam.ac.in

BONAFIDE CERTIFICATE

Bonafide record of work done by _____

of_____in _____

during odd semester in academic year 2026-2027.

Staff In-charge

Submitted to the practical Examination held at Kalasalingam Academy of Research and Education, Anand Nagar, Krishnankoil on _____

REGISTER NUMBER										

Internal Examiner

External Examiner

EXPERIMENT EVALUATION SUMMARY

Name: **Reg No:**

S.No	Date	Experiment	Marks (10)	Faculty Signature

INTRODUCTION TO OPERATING SYSTEMS

OBJECTIVE:

This lab complements the operating systems course. Students will gain practical experience with designing and implementing concepts of operating systems such as system calls, CPU scheduling, process management, memory management, file systems and deadlock handling and disk scheduling using C language in Linux environment.

OUTCOMES:

Upon the completion of Operating Systems practical course, the student will be able to:

1. **Understand** and implement basic services and functionalities of the operating system using system calls.
2. **Use** modern operating system calls and synchronization libraries in software/ hardware interfaces**.**
3. **Understand** the benefits of thread over process and implement synchronized programs using multithreading concepts.
4. **Analyze** and simulate CPU Scheduling Algorithms like FCFS, Round Robin, SJF, and Priority.
5. **Implement** memory management schemes and page replacement schemes.
6. **Simulate** file allocation and organization techniques.
7. **Understand** the concepts of deadlock in operating systems and implement them in multiprogramming system.

BASIC LINUX COMMANDS

EX. No. 1a WORKING WITH FILES AND DIRECTORIES

AIM:

To study the UNIX commands for accessing files and directories.

COMMANDS DESCRIPTION:

- cat --- for creating and displaying short files
- chmod --- change permissions
- cd --- change directory
- cp --- for copying files
- date --- display date
- echo --- echo argument
- ftp --- connect to a remote machine to download or upload files
- grep --- search file
- head --- display first part of file
- ls --- see what files you have
- lpr --- standard print command
- more --- use to read files
- mkdir --- create directory
- mv --- for moving and renaming files
- ncftp --- especially good for downloading files via anonymous ftp.
- print --- custom print command
- pwd --- find out what directory you are in
- rm --- remove a file
- rmdir --- remove directory
- rsh --- remote shell
- setenv --- set an environment variable
- sort --- sort file
- tail --- display last part of file
- tar --- create an archive, add or extract files
- telnet --- log in to another machine
- wc --- count characters, words, lines

EXECUTION & OUTPUT:

Assessment of an Individual experiment		
Algorithm	**2**	
Program	**4**	
Output	**2**	
Viva	**2**	
Total	**10**	
Faculty Signature		

Result:

The general purpose utility commands was executed successfully.

Ex. No. 1b **General Purpose Utility Commands**

AIM:

To work with some of the general-purpose utility commands in UNIX.

COMMANDS DESCRIPTION:

- date - print or set the system date and time
- bc - An arbitrary precision calculator language
- echo - Display a line of text.
- printf - Format and print data
- passwd - update user's authentication tokens
- who - Show who is logged on
- who am I - Show who is logged on and what they are doing
- uname - Print system information
- expr - Evaluate expressions
- test - Check file types and compare values
- seq - Print a sequence of numbers
- factor - Factor numbers
- rev - reverse lines of a file or files
- cal - Displays a calendar
- date - print or set the system date and time
- bc - An arbitrary precision calculator language
- echo - Display a line of text.
- printf - Format and print data
- passwd - update user's authentication tokens
- who - Show who is logged on
- w - Show who is logged on and what they are doing
- uname - Print system information
- expr - Evaluate expressions
- test - Check file types and compare values
- seq - Print a sequence of numbers
- factor - Factor numbers
- rev - reverse lines of a file or files

EXECUTION & OUTPUT:

Assessment of an Individual experiment		
Algorithm	**2**	
Program	**4**	
Output	**2**	
Viva	**2**	
Total	**10**	
Faculty Signature		

Result

The general purpose utility commands was executed successfully.

SYSTEM CALLS

Ex. No. 2a **Fork System Call**

INSTRUCTIONS:

System call fork() is used to create processes. It takes no arguments and returns a process ID. The purpose of fork() is to create a new process, which becomes the child process of the caller. After a new child process is created, both processes will execute the next instruction following the fork() system call. Therefore, we have to distinguish the parent from the child. This can be done by testing the returned value of fork():

- If fork() returns a negative value, the creation of a child process was unsuccessful.
- fork() returns a zero to the newly created child process.
- fork() returns a positive value, the process ID of the child process, to the parent. The returned process ID is of type pid_t defined in sys/types.h. Normally, the process ID is an integer. Moreover, a process can use function getpid() to retrieve the process ID assigned to this process.

AIM:

To write a program to create a child process using the system calls –fork.

ALGORITHM:

1. fork () to create a child process from the main/parent process. The fork () system function does not accept any argument. It returns an integer of the type pid_t. On success, fork () returns the PID of the child process which is greater than 0. Inside the child
Process, the return value is 0. If fork () fails, then it returns -1.
2. Print the PID (Process ID) and PPID (Parent Process ID) from child and parent process.
3. On the parent process wait(NULL) is used to wait for the child process to finish
4. On the child process, exit() is used to finish the child process

PROGRAM:

Assessment of an Individual experiment		
Algorithm	**2**	
Program	**3**	
Output	**3**	
Viva	**2**	
Total	**10**	
Faculty Signature		

Result

Hence create a child process using the system calls – fork executed successfully

Ex. No. 2b **EXEC System Call**

DESCRIPTION:

EXEC is a functionality of an operating system that runs an executable file in the context of an already existing process, replacing the previous executable. This act is also referred to as an overlay. It is especially important in Unix-like systems, although exists elsewhere. As a new process is not created, the process identifier (PID) does not change, but the machine code, data, heap, and stack of the process are replaced by those of the new program.

AIM:

To write a program to display time and date using exec system call.

ALGORITHM:

1. Current process image is overwritten with a new process image.
2. New process image is the one you passed as exec argument
3. The currently running process is ended
4. New process image has same process ID, same environment, and same file descriptor (because process is not replaced process image is replaced)
5. The CPU stat and virtual memory is affected. Virtual memory mapping of the current process
image is replaced by virtual memory of new process image.
6. execlp("date","date", NULL); //uses the PATH to find date, try: %echo $PATH

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result:

Thus to display time and date using exec system call was executed successfully.

Ex.No. 2c **STAT System Call**

DESCRIPTION:

stat() is a Unix system call that returns file attributes about an node. The semantics of stat() vary between operating systems. As an example, Unix command ls uses this system call to retrieve information on files that includes:

- atime: time of last access (ls -lu)
- mtime: time of last modification (ls -l)
- ctime: time of last status change (ls -lc)

AIM:

To write a program to implement STAT system call.

ALGORITHM:

i. These functions return information about a file, in the buffer pointed to by statbuf.
ii. No permissions are required on the file itself, butin the case of stat(), fstatat(), and lstat()execute (search) permission is required on all of the directories in pathname that lead to the filestat() and fstatat() retrieve information about the file pointed to by pathname; the differences for fstatat() are described below.
lstat() is identical to stat(), except that if pathname is a symbolic link, then it returns information about the link itself, not the file that the link refers to.fstat() is identical to stat(), except that the file about which information is to be retrieved is specified by the file descriptor fd.
iii. The stat structure All of these system calls return a stat structure, which contains the following fields:

```
struct stat {
dev_t st_dev;          /* ID of device containing file */
ino_t st_ino;          /* Inode number */
mode_t st_mode;    /* File type and mode */
nlink_t st_nlink;    /* Number of hard links */
uid_t st_uid;        /* User ID of owner */
gid_t st_gid;        /* Group ID of owner */
dev_t st_rdev;              /* Device ID (if special file) */
```

```
off_t st_size;            /* Total size, in bytes */
blksize_t st_blksize;          /* Block size for filesystem I/O */
blkcnt_t st_blocks;  /* Number of 512B blocks allocated */
```

/* Since Linux 2.6, the kernel supports nanosecond precision for the following timestamp fields. For the details before Linux 2.6, see NOTES. */

```
struct timespec st_atim;          /* Time of last access */
struct timespec st_mtim;         /* Time of last modification */
struct timespec st_ctim;       /* Time of last status change */
#define st_atime st_atim.tv_sec  /* Backward compatibility */
#define st_mtime st_mtim.tv_sec
#define st_ctime st_ctim.tv_sec
```

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result:

Thus program to implement STAT system call was implemented successfully.

Ex. No. 2d **Wait System Call**

DESCRIPTION:

The system call wait() is blocks the calling process until one of its child processes exits or a signal is received. For our purpose, we shall ignore signals. Wait() takes the address of an integer variable and returns the process ID of the completed process. Some flags that indicate the completion status of the child process are passed back with the integer pointer. One of the main purposes of wait() is to wait for completion of child processes.

The execution of wait() could have two possible situations.

1. If there are at least one child processes running when the call to wait() is made, the caller will be blocked until one of its child processes exits. At that moment, the caller resumes its execution.
2. If there is no child process running when the call to wait() is made, then this wait() has no effect at all. That is, it is as if no wait() is there.

AIM:

To write a program using wait system call.

ALGORITHM:

1. print before fork. Then fork () system call creates a child process.
2. Wait () system call is added to the parent section of the code.
3. Hence, the moment processor starts processing the parent, the parent process is suspended because the very first statement is wait (NULL).
4. Thus, first, the child process runs, and the output lines are all corresponding to the child process.

Once the child process finishes, parent resumes and prints all its printf () statements.

5. The NULL inside the wait () means that we are not interested to know the status of change of state of child process

PROGRAM

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Thus program using wait system call was executed successfully

Ex. No. 2e **Input Output system Call**

DESCRIPTION:

- creat(name, permissions) – Used to create a file with the name and mode specified. Here, permission would be a number. 0666 means read write permissions.

- open(name, mode) – Used to open a file name in the mode (read or write) specified. 0 is for opening in read mode, 1 for writing and 2 for both.

- close(fd) – Close a opened file.

- unlink(fd) – Delete a file.

- read(fd, buffer, n_to_read) – Read data from a file.

- write(fd, buffer, n_to_write) - write data from to a file.

- lseek(fd, offest, whence) - Move the read/write pointer to the specified

location. AIM:

To write a program to implement the concept of I/O call.

ALGORITHM:

1. Create new empty file on disk
2. Create file table entry
3. Set first unused file descriptor to point to file table entry
4. Return file descriptor used, -1 upon failure
5. open: Used to Open the file for reading, writing or both.

Syntax in C language

```
#include<sys/types.h>
#include<sys/stat.h>
#include <fcntl.h>
```

6. int open (const char* Path, int flags [, int mode]);

Parameters

Path : path to file which you want to use absolute path begin with /, when you are not work in same directory of file.

Use relative path which is only file name with extension, when you are work in same directory of file.
flags : How you like to use
O_RDONLY: read only, O_WRONLY: write only, O_RDWR: read and write, O_CREAT: create
file if it doesnt exist, O_EXCL: prevent creation if it already exists
How it works in OS

7. Find existing file on disk
8. Create file table entry
9. Set first unused file descriptor to point to file table entry
10. Return file descriptor used, -1 upon failure

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Thus program to implement the concept of I/O call was executed successfully.

SCHEDULING ALGORITHMS

Ex. No. 3a **First Come First Serve Scheduling**

AIM:

To write a program using C in UNIX environment to implement the first come first serve scheduling.

ALGORITHM:

Step1: Create the number of processes.

Step2: Get the ID and Service time for each process.

Step3: Initially, waiting time of first process is zero and Total time for the first process is the starting time of that process.

Step4: Calculate the Total time and Processing time for the remaining processes.

Step5: Waiting time of one process is the Total time of the previous process.

Step6: Total time of process is calculated by adding Waiting time and Service time.

Step7: Total waiting time is calculated by adding the waiting time for lack process.

Step8: Total turnaround time is calculated by adding all total time of each process.

Step9: Calculate Average waiting time by dividing the total waiting time by total number of process.

Step10: Calculate Average turnaround time by dividing the total time by the number of process.

Step11: Display the result

PROGRAM

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result:

Thus the C program to implement CPU scheduling algorithm for first come first serve was executed successfully.

Ex. No. 3b **Shortest Job First Scheduling**

AIM:

To write a program using C in UNIX environment to implement the shortest job next
concept.

ALGORITHM:

Step1: Get the number of processes.

Step2: Get the id and service time for each process.

Step3: Initially the waiting time of first short process as 0 and total time of first short is process the service time of that process.

Step4: Calculate the total time and waiting time of remaining process.

Step5: Waiting time of one process is the total time of the previous process.

Step6: Total time of process is calculated by adding the waiting time and service time of each process.

Step7: Total waiting time calculated by adding the waiting time of each process.

Step8: Total turnaround time calculated by adding all total time of each process.

Step9: Calculate average waiting time by dividing the total waiting time by total number of Processes.

Step10: Calculate average turnaround time by dividing the total waiting time by total number of process.

Step11: Display the result

PROGRAM:

Result

Thus the C program to implement the CPU scheduling algorithm for shortest job first was executed successfully.

Ex. No. 3c **Round Robin Scheduling**

AIM:

To write a program using C in UNIX environment to implement Round-robin scheduling concept.

ALGORITHM:

1) Receive inputs from the user to fill process id,burst time and arrival time.
2) Calculate the waiting time for all the process id.
   i) The waiting time for first instance of a process is calculated as:
   a[i].waittime=count + a[i].arrivt
   ii) The waiting time for the rest of the instances of the process is calculated as:
a) If the time quantum is greater than the remaining burst time then waiting time is calculated as:
   a[i].waittime=count + tq
b) Else if the time quantum is greater than the remaining burst time then waiting time is Calculated as:
   a[i].waittime=count - remaining burst time
3) Calculate the average waiting time and average turnaround time
4) Print the result

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result:

Thus the C program to simulate CPU scheduling algorithm for round robin was executed successfully.

Ex.No. 3d **Priority Scheduling**

AIM:

To implement the priority scheduling using C in UNIX environment.

ALGORITHM:

Step 1: Inside the structure declare the variables.
Step 2: Declare the variable i,j as integer, totwtime and totttime is equal to zero.
Step 3: Get the value of n assign p and allocate the memory.
Step 4: Inside the for loop get the value of burst time and priority.
Step 5: Assign wtime as zero .
Step 6: Check p[i].pri is greater than p[j].pri .
Step 7: Calculate the total of burst time and waiting time and assign as turnaround time.
Step 8: Stop the program.

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Thus the C program to implement the priority scheduling algorithm was executed successfully.

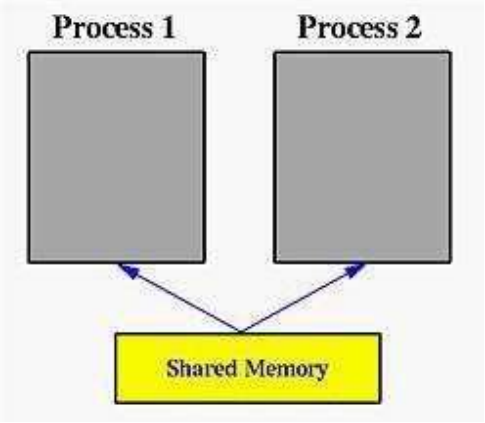
Ex.No : 4 SIMULATION of IPC in UNIX

AIM:

To write a C program to implement inter process communication.

DESCRIPTION:

In the discussion of the **fork()** system call, we mentioned that a parent and its children have separate address spaces. While this would provide a more secured way of executing parent and children processes (because they will not interfere each other), they shared nothing and have no way to communicate with each other. A ***shared memory*** is an extra piece of memory that is *attached* to some address spaces for their owners to use. As a result, all of these processes share the same memory segment and have access to it. Consequently, race conditions may occur if memory accesses are not handled properly. The following figure shows two processes and their address spaces. The yellow rectangle is a shared memory attached to both address spaces and both process 1 and process 2 can have access to this shared memory as if the shared memory is part of its own address space. In some sense, the original address spaces is "extended" by attaching this shared memory.



Shared memory is a feature supported by UNIX System V, including Linux, SunOS and Solaris. One process must explicitly ask for an area, using a ***key***, to be shared by other processes. This process will be called the *server*. All other processes, the *clients*, that know the shared area can access it. However, there is no protection to a shared memory and any

process that knows it can access it freely. To protect a shared memory from being accessed at the same time by several processes, a synchronization protocol must be setup.

A shared memory segment is identified by a unique integer, the ***shared memory ID***. The shared memory itself is described by a structure of type **shmid_ds** in header file **sys/shm.h**. To use this file, files **sys/types.h** and **sys/ipc.h** must be included.

ALGORITHM:

1. Writer requests the entry to critical section.
2. If allowed i.e. wait() gives a true value, it enters and performs the write. If not allowed, it keeps
on waiting.
3. It exits the critical section.

```
do {
// writer requests for critical section wait(wrt);
// performs the write
// leaves the critical section signal(wrt);
}
while(true);
```

PROGRAM:

SHARED MEMORY FOR READER PROCESS

ALGORITHM:

1) Reader requests the entry to critical section.
2) If allowed:

2.1) it increments the count of number of readers inside the critical section. If this reader is the first reader entering, it locks the wrt semaphore to restrict the entry of writers if any reader is inside.
2.2) It then, signals mutex as any other reader is allowed to enter while others are already reading.
2.3) After performing reading, it exits the critical section. When exiting, it checks if no more reader is inside, it signals the semaphore wrt as now, writer can enter the critical section.

3) If not allowed, it keeps on waiting.

```
do {
// Reader wants to enter the critical section
wait(mutex);
// The number of readers has now increased by 1 readcnt++;
// there is atleast one reader in the critical section
// this ensure no writer can enter if there is even one reader
// thus we give preference to readers here if (readcnt==1) wait(wrt);
// other readers can enter while this current reader is inside
// the critical section signal(mutex);
// current reader performs reading here wait(mutex);
// a reader wants to leave readcnt--;
// that is, no reader is left in the critical section, if (readcnt == 0) signal(wrt); // writers can
enter signal(mutex);
// reader leaves } while(true);
```

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

To write a C program to implement inter process communication was implemented successfully.

Ex.No: 5 IMPLEMENTATION OF DEADLOCK - BANKERS ALGORITHM

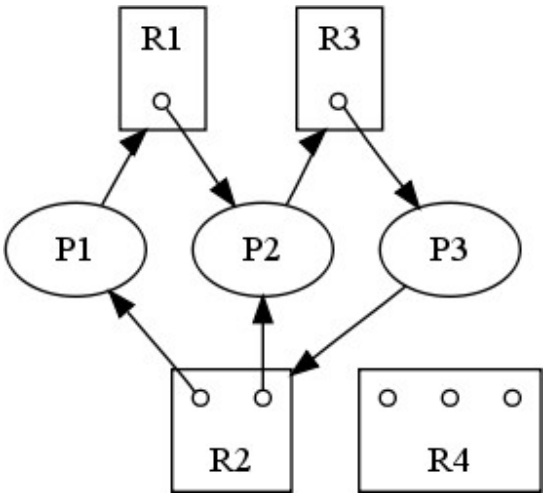
AIM:

To write a UNIX C Program for the Implementation of Banker's

PROBLEM DESCRIPTION:

The Banker's Algorithm was designed and developed by a Dutch Computer Scientist, Edsger Dijkstra. The Banker's Algorithm is a Resource Allocation and a Deadlock Avoidance Algorithm.

This algorithm takes analogy of an actual bank where clients request to withdraw cash. The Banking Authorities have some data according to which the cash is lent to the client. The Banker cannot give more cash than the client's request and the total cash available in the bank.



The Banker's Algorithm is divided into two parts:

1. Safety Test Algorithm: This algorithm checks the current state of the system to maintain its Safe State.

2. Resource Request Handling Algorithm: This algorithm verifies if the requested

resources, after their allocation to the processes affects the Safe State of the System. If it does, then the request of the process for the resource is denied, thereby maintaining the Safe State.

A State is Safe if it is possible for all the Processes to Complete its Execution without causing any Deadlocks. An Unsafe State is the one in which the Processes cannot complete its execution.

ALGORITHM

Step-1: Start the program.
Step-2: Declare the memory for the process.
Step-3: Read the number of processes, resources, allocation matrix and available matrix.
Step-4: Compare each process using the banker's algorithm.
Step-5: If the process is in safe state, then it is a not a deadlock process otherwise it is a deadlock process
Step-6: produce the result of state of process
Step-7: Stop the program

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Thus the Banker's algorithm was implemented successfully.

Ex. No:6 PAGE REPLACEMENT ALGORITHMS

PROBLEM DESCRIPTION:

Page replacement algorithms are used to decide what pages to page out when a page needs to be allocated. This happens when a page fault occurs and free page cannot be used to satisfy allocation

LRU:

"Replace *the page that had not been used* for a longer sequence of time". The frames are empty in the beginning and initially no page fault occurs so it is set to zero. When a page fault occurs the page reference sting is brought into the memory. The operating system keeps track of all pages in the memory, thereby keeping track of the page that had not been used for longer sequence of time. If the page in the page reference string is not in memory, the page fault is incremented and the page that had not been used for a longer sequence of time is replaced. If the page in the page reference string is in the memory take the next page without calculating the next page. Take the next page in the page reference string and check if the page is already present in the memory or not. Repeat the process until all pages are referred and calculate the page fault for all those pages in the page references string for the number of available frames.

Ex. No:6a LRU algorithm

AIM : To write c program to implement LRU algorithm.

Algorithm

- Declare the size.
- Get the number of pages to be inserted.
- Get the value.
- Declare counter and stack.
- Select the least recently used page by counter value.
- Stack them according the selection.
- Display the values.Stop the process

Program

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Thus c program to implement LRU algorithm was implemented successfully.

Ex. No: 6b **Optimal replacement algorithm**

AIM : **:** To write c program to implement Optimal replacement algorithm

ALGORITHM:

1. Read The Number of Pages, Reference String, Cache Size
2. Let Cache Size Be N. Add First N Elements from Reference String to Cache Directly. Print Cache And
Increment Page Faults.
3. For The Elements Left Do the Following
    3.1 There May Be 2 Cases. Either The Element in The Reference String May Be In The Cache Already. Or It Is Not Present
    3.2 If The Element Is Not In The Cache Do The Following (Uses The Function Ispresent() To Check If The Element Is Present or Not)
    3.2.1 For All the Elements in The Cache Find Respective Length From Current Index In The Reference String (Using Findlength() Function)
    3.2.2 Find The Maximum Of The Lengths (Using Findmax() Function)
    3.2.3 Replace That Index Of Cache With The Element Of Reference String
    3.2.4 Increment Page Fault
  3.3 Else Do Nothing. Print Page Faults.

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

RESULT : Thus c program to implement Optimal replacement algorithm was implemented successfully.

Ex. No: 6c **FIFO algorithm**

AIM : To write c program to implement FIFO algorithm

ALGORITHM:
1. Start the process
2. Declare the size with respect to page length
3. Check the need of replacement from the page to memory
4. Check the need of replacement from old page to new page in memory
5. Forma queue to hold all pages
6. Insert the page require memory into the queue
7. Check for bad replacement and page fault
8. Get the number of processes to be inserted
9. Display the values
10. Stop the process

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result : Thus c program to implement FIFO algorithm was implemented successfully.

MEMORY MANAGEMENT FUNCTIONS

Ex. No. 7a Memory Management Scheme using Best Fit.

AIM:

To write a C program to implement the memory management scheme using best fit.

ALGORITHM:

1. Get no. of Processes and no. of blocks.
2. After that get the size of each block and process requests.
3. Then select the best memory block that can be allocated using the above definition.
4. Display the processes with the blocks that are allocated to a respective process.
5. Value of Fragmentation is optional to display to keep track of wasted memory.
6. Stop.

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result : Thus c program to implement best fit was executed successfully.

Ex. No. 7b **Memory Management Scheme using First Fit.**

AIM:

To write a C program to implement the memory management scheme using first fit

ALGORITHM:

1. Get no. of Processes and no. of blocks.
2. After that get the size of each block and process requests.
3. Now allocate processes

if(block size >= process size)
//allocate the process
else
//move on to next block

4. Display the processes with the blocks that are allocated to a respective process.
5. Stop.

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Thus c program to implement best fit was implemented successfully.

Ex. No. 7c Memory Management Scheme using Worst Fit.

AIM:

To write a C program to implement the memory management scheme using worst fit.

ALGORITHM:

Step 1: Input memory block with a size.
Step 2: Input process with size.
Step 3: Initialize by selecting each process to find the maximum block size that can be assigned to
the current process.
Step 4: If the condition does not fulfil, they leave the process.
Step 5: If the condition is not fulfilled, then leave the process and check for the next process.
Step 6: Stop.

PROGRAM:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Thus c program for worst fit was executed successfully

Ex No: 8 IMPLEMENTATION OF DISK SCHEDULING

ALGORITHMS AIM:

To write a 'C' program to implement the Disk Scheduling algorithm for First Come First
Served (FCFS), Shortest Seek Time First (SSTF), and SCAN.

PROBLEM DESCRIPTION:

Disk Scheduling is the process of deciding which of the cylinder request is in the ready queue is to be accessed next.

The access time and the bandwidth can be improved by scheduling the servicing of disk I/O requests in good order.

Access Time:

The access time has two major components: **Seek time and Rotational Latency.**

Seek Time:

Seek time is the time for disk arm to move the heads to the cylinder containing the desired sector.

Rotational Latency:

Rotational latency is the additional time waiting for the disk to rotate the desired sector to the disk head.

Bandwidth:

The disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.

PROCEDURE:

1. Input the maximum number of cylinders and work queue and its head starting position.
2. **First Come First Serve Scheduling (FCFS) algorithm** – The operations are performed in order requested
3. There is no reordering of work queue.
4. Every request is serviced, so there is no starvation
5. The seek time is calculated.

6. **Shortest Seek Time First Scheduling (SSTF) algorithm** – This algorithm selects the request with the minimum seek time from the current head position.

7. Since seek time increases with the number of cylinders traversed by the head, SSTF chooses the pending request closest to the current head position.

8. The seek time is calculated.

9. **SCAN Scheduling algorithm** –The disk arm starts at one end of the disk, and moves toward the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk.

10. At the other end, the direction of head movement is reversed and servicing continues.

11. The head continuously scans back and forth across the disk.

12. The seek time is calculated.

13. Display the seek time and terminate the program.

Program:

SSTF:

ALORITHM:

1. Let Request array represents an array storing indexes of tracks that have been requested. head is the position of disk head.
2. Find the positive distance of all tracks in the request array from head.
3. Find a track from requested array which has not been accessed/serviced yet and has minimum distance from head.
4. Increment the total seek count with this distance.
5. Currently serviced track position now becomes the new head position.
6. Go to step 2 until all tracks in request array have not been serviced

PROGRAM

SCAN:

ALGORITHM:

1. Let Request array represents an array storing indexes of tracks that have been requested in ascending order of their time of arrival. head is the position of disk head.
2. Let direction represents whether the head is moving towards left or right.
3. In the direction in which head is moving service all tracks one by one.
4. Calculate the absolute distance of the track from the head.
5. Increment the total seek count with this distance.
6. Currently serviced track position now becomes the new head position.

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Thus implemented disk scheduling algorithms.

Ex No: 9 IMPLEMENTATION OF ACCESS CONTROL

MECHANISM AIM:

To write a 'C' program to implement various Access Control Mechanism in Operating Systems.

Access control is the traditional center of gravity of computer security. It is where security engineering meets computer science. Its function is to control which principals (persons, processes, machines, . . .) have access to which resources in the system—which files they can read, which programs they can execute, how they share data with other principals, and so on.

The access control mechanisms, which the user sees at the application level, may express a very rich and complex security policy. A modern online business could assign staff to one of dozens of different roles, each of which could initiate some subset of several hundred possible transactions in the system. Some of these (such as credit card transactions with customers) might require online authorization from a third party while others (such as refunds) might require dual control.The applications may be written on top of middleware, such as a database management system or bookkeeping package, which enforces a number of protection properties. For example, bookkeeping software may ensure that a transaction that debits one ledger for a certain amount must credit another ledger for the same amount

The middleware will use facilities provided by the underlying operating system. As this constructs resources such as files and communications ports from lower-level components, it acquires the responsibility for providing ways to control access to them.

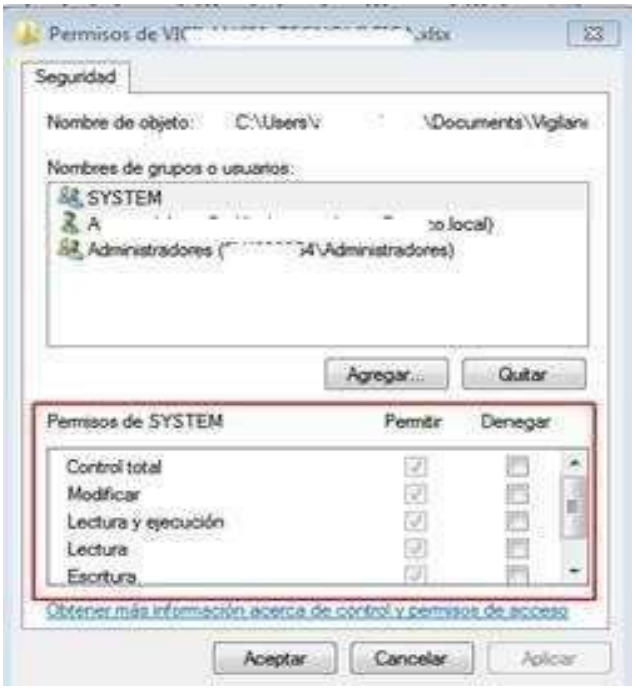
Finally, the operating system access controls will usually rely on hardware features provided by the processor or by associated memory management hardware. These control which memory addresses a given process can access

Types of Access Control.

Based on the restrictions and access control to systems, we come across three main types: discretionary access control (DAC), role based access (RBAC) and mandatory access control (MAC).

Discretionary Access control, DAC: Discretionary access control is a means of restricting access to objects based on the identity of subjects that try to operate or access them. The most representative example is the mechanism of permissions established by the owner (user/group) of the subject. Therefore, it's the owner of the object who determines which users and with which privileges they access his/her objects. This access mechanism is present

in the majority of operating systems. In this type of access control its normal to use attributes (read, write, execute, etc) to mark the permissions applied to the object. In the same way, there is an owner/user and a set of options that can be shared (groups, other users…).



DAC permissions in a Windows file -

In the particular case of UNIX/Linux systems, the objects of the system (files, directories, etc) include three attributes: read (r), write (w) and execute (x). These permissions are assigned by the user for the group and others (users that are not owners and don't belong the group).

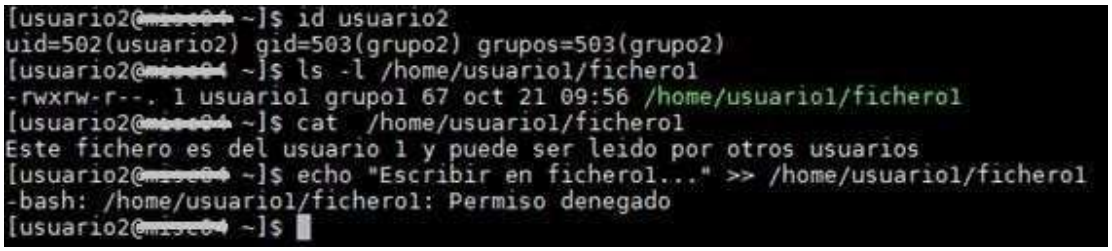
In Linux, with the ls -l command we can visualize the basic DAC permissions. In the following example we can see a system, where user1, who belongs to group1, has fixed the read/write/execute permissions **(rwx)** to himself as the owner. The users that belong to the same group (grupo1) have the same read and write permissions **(rw-)** and any other user only has a read permission **(r--)**:

```
[usuario1@ ~]$ ls -l
total 0
-rwxrw-r--. 1 usuario1 grupo1 0 oct 21 09:45 fichero1
[usuario1@ ~]$
```

*Discretional access permissions (DAC), traditional in *NIX systems -*

This way, any read, write or execute action that doesn't comply with these permission will be

denied.



- usuario2 can read but not modify usuario1's fichero1 because of the assigned permissions -

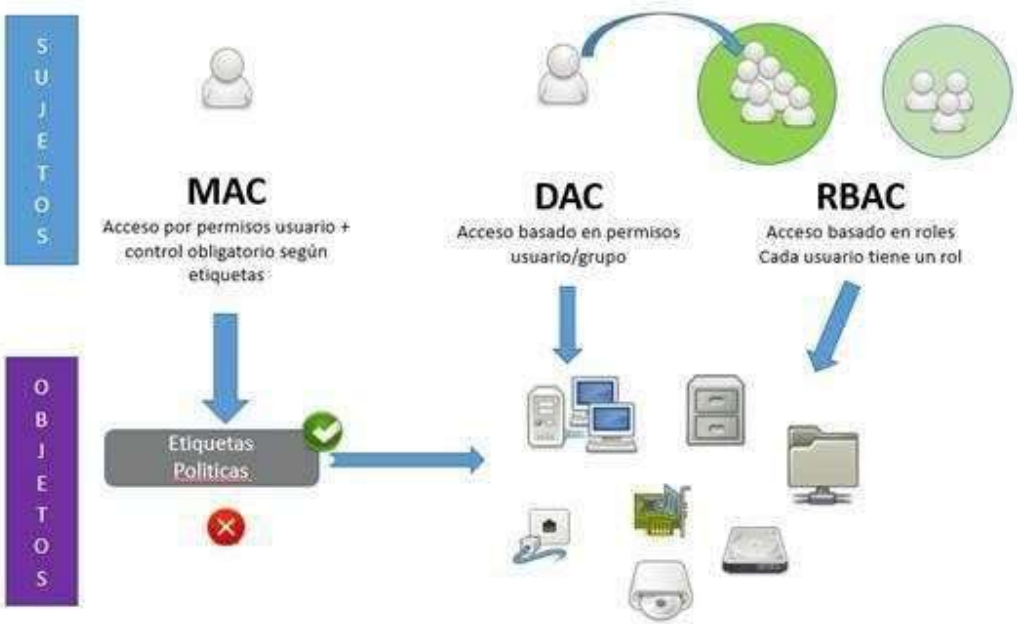
Other properties exist such as ACLs (access control list) or other a href="http://en.wikipedia.org/wiki/File_system_permissions" rel="external">special permissions such as sticky bit, or setuid, setgid (Linux) permissions that add more options to the DAC but are out of the introductory reach of this part.

Role-Based Access Control (RBAC)

Discretional access controls don't provide a sufficient granularity to enable a more defined and structured segmentation in a complex system with multiple users and functions. In this case, a role mechanism offers greater versatility. Role-based access control consists in the definition of roles that have been attributed a number of characteristics applied to the permissions and actions that they can carry out, including controlling other roles. It is, in a way, a hierarchical system of classes. Often used in organizations with a great number of users where different work groups or departments with different functions are integrated, such as for example systems, development, commercial, general service departments. With this mechanism, access to objects and tasks can be efficiently segmented and organized. Notable cases of these mechanisms are LDAP, Active Directory of Microsoft Windows or FreeIPA of Fedora/Redhat. Some UNIX systems such as Solaris or AIX all implement this system of privileges.

Mandatory Access Control, MAC: This access mechanism is a compliment of the previous ones and adds another safety layer for access and privilege control. MAC bases itself on "tagging" every element in the system that will then undergo the access control policies that

have been configured. Therefore, in any operation by a subject on an object the tags will be verified and the established MAC policies will be applied to determine if the operation is allowed, even when it has complied with other security controls. In contrast with the discretionary access control, the user can modify permissions and tags but can't fix access controls that suppose a violation of the system's MAC policies. It's the responsibility of a privileged user to establish the MAC's central policies that will govern the controls to be applied depending on the established tags. Examples of these control methods are SELinux in Linux/Redhat/Centos/Fedora distributions or AppArmor in Linux SuSE.



Types of access control

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Access Control Mechanism have been executed successfully.

Ex No: 10 IMPLEMENTATION OF FILE SYSTEM

AIM:

To write a 'C' program to implement various file handling operations

Reading from a File

The file read operation in C can be performed using functions fscanf() or fgets(). Both the functions performed the same operations as that of scanf and gets but with an additional parameter, the file pointer. There are also other functions we can use to read from a file. Such functions are listed below:

Function	Description
fscanf()	Use formatted string and variable arguments list to take input from a file.
fgets()	Input the whole line from the file.
fgetc()	Reads a single character from the file.
fgetw()	Reads a number from a file.
fread()	Reads the specified bytes of data from a binary file.

Write to a File

The file write operations can be performed by the functions fprintf() and fputs() with similarities to read operations. C programming also provides some other functions that can be used to write data to a file such as:

Function	Description
fprintf()	Similar to printf(), this function use formatted string and variable arguments list to print output to the file.
fputs()	Prints the whole line in the file and a newline at the end.
fputc()	Prints a single character into the file.

fputw()	Prints a number to the file.
fwrite()	These functions write the specified number of bytes to the binary file.

Implementation:

Assessment of an Individual experiment		
Algorithm	2	
Program	3	
Output	3	
Viva	2	
Total	10	
Faculty Signature		

Result

Thus, the file handling operations have been executed successfully.